

# SatQuery AI: Vision-Language Assistant for Multimodal Remote Sensing Image Analysis

**Problem Statement ID:** SIH26167 | **Organization:** Indian Space Research Organisation (ISRO)

**Theme:** Space Technology | **Track:** Software Track

---

## Executive Summary

Traditional earth observation AI structures suffer from static operational limitations, operating purely as isolated pipelines configured for singular predefined tasks (e.g., land-cover categorization or standalone object bounding). **SatQuery AI** overrides this bottleneck by introducing an enterprise-ready, interactive, Multi-Modal Vision-Language Framework tailored explicitly for geospatial analytics. The pipeline translates natural-language queries dynamically into georeferenced visual output layers (GeoJSON and spatial matrices) through cross-attention fusion heads mapping Optical RGB, Synthetic Aperture Radar (SAR), and Bi-Temporal imagery arrays natively.

## Architectural Pipeline Comparison Matrix

| Pipeline Phase       | Legacy Systems / Static VLMs                                          | SatQuery Next-Gen Framework                                            |
|----------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------|
| Ingestion Proxy      | Hardcoded single-sensor array structures; raw band-truncation errors. | Asynchronous Multi-Modal Broker preserving full spectral layers.       |
| Query Interpretation | Generic LLM token prediction; high geometric hallucination risks.     | Intent-Driven Dynamic Router partitioning workflow specific pipelines. |
| Visual Grounding     | Coarse coordinate text tokens bounding loose general areas.           | Cascaded Deep Feature Head feeding bounding targets into SAM 2.        |
| Output Presentation  | Raw chatbot string responses requiring secondary GIS mapping.         | Production-grade GeoJSON vector files mapped to EPSG:4326 natively.    |

## Chapter 1: Multi-Modal Geospatial Ingestion Pipeline

The system leverages rasterio and GDAL proxies to parse deep-layer satellite GeoTIFF matrices asynchronously. Instead of squashing imagery down to standard 8-bit 3-channel RGB dimensions, the input proxy securely retains native metadata tables, multi-spectral band stacks (e.g., SWIR, NIR), and explicit Coordinate Reference Systems (CRS) tags.

## Chapter 2: Intent-Driven Agentic Routing System

User queries are broken down by a tokenized intent-classifier network. Queries containing terms like 'flood', 'microwave', or 'radar' are dispatched down the SAR specialist stream. If the prompt targets time-series variations ('growth', 'before/after'), the Temporal stream structures joint-feature pairs automatically.

## Chapter 3: Cross-Modal Attention & ML Core Model Layout

The technical core integrates a custom GeospatialCrossAttention block configured to align raw optical tokens directly against radar backscatter distributions prior to feeding the unified multimodal LLVM matrix decoder.

```
import torch
import torch.nn as nn

class GeospatialCrossAttention(nn.Module):
    def __init__(self, feature_dim: int = 512, num_heads: int = 8):
        super(GeospatialCrossAttention, self).__init__()
        self.mha = nn.MultiheadAttention(embed_dim=feature_dim, num_heads=num_heads, batch_first=True)
        self.ln_opt = nn.LayerNorm(feature_dim)
        self.ln_sar = nn.LayerNorm(feature_dim)
        self.ffn = nn.Sequential(
            nn.Linear(feature_dim, feature_dim * 4),
            nn.ReLU(),
            nn.Linear(feature_dim * 4, feature_dim)
        )
        self.final_norm = nn.LayerNorm(feature_dim)

    def forward(self, optical_feats, sar_feats):
        norm_opt = self.ln_opt(optical_feats)
        norm_sar = self.ln_sar(sar_feats)
        attn_out, _ = self.mha(query=norm_opt, key=norm_sar, value=norm_sar)
        interim = optical_feats + attn_out
        return interim + self.ffn(self.final_norm(interim))
```

## Chapter 4: Cascaded Visual Grounding Head (SAM 2 Integration)

Coordinates provided by the vision language core are linked as prompt anchors to a SAM2ImagePredictor pipeline. This cascade steps around textual formatting limitations, turning text strings into accurate, pixel-level binary object boundaries.

## Chapter 5: Geospatial Vector Engine & GeoJSON Projector

Extracted contours are parsed via OpenCV and re-projected using the original spatial transform metadata. Pixels convert directly into real-world geographic coordinates, compiled instantly as standard GeoJSON FeatureCollections.

## Chapter 6: Verifier Audit & Self-Correction Loop

The verifier pipeline compares generated vector polygon outputs against baseline GIS topology registries. Detections breaching logical geospatial constraints (such as temporary building arrays masking over deep open water layers) are flagged and suppressed to completely block VLM hallucinations.

## Chapter 7: Production Dataset Integration Matrix

Training utilizes pre-aligned remote-sensing vision-language configurations: **RSVLM-QA** for conversational spatial analysis (~162,000 question-answer matrices), **VRSBench** for text-anchored object coordinate detection, and **FloodNet** for radar-driven UAV/satellite structural damage evaluations.

---

*This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.*